MIT 8103 Advanced Database Systems: Portfolio Documentation

Student: Moyinoluwa Toluwase Agbo

Programme: Master of Information Technology, Miva Open University (September 2026 cohort);
GitHub repository (private): https://github.com/moyinoluwaagbo/advanced-database-portfolio

1. Selected case study: MivaDB

- **Organisation/scenario:** MivaDB, a simulated student management and results system modelled on Miva Open University.
- **Nature of the system:** an online university that registers students into programmes and courses each semester, records their results, and delivers teaching through an LMS.
- **Users:** students, lecturers, programme coordinators and registry staff.
- **Major data:** programmes, lecturers, semesters, courses, students, course offerings, enrolments and results (relational data), and LMS activity events such as logins, video views, quiz attempts, forum posts and assignment submissions (document data).
- **Key operations:** register students for course offerings within seat limits, enter and look up results, report average scores per course, and record and query LMS activity.

All five portfolio activities use this same case study. All data is synthetic, generated with Faker and my own Python scripts.

2. Summary of the five portfolio activities

Portfolio 1: Database design and modelling. I designed a PostgreSQL schema for the eight entities above, with primary keys, foreign keys and constraints, and produced an ER diagram in pgAdmin. The normalisation write-up takes a flat table through 1NF, 2NF and 3NF and shows the update, insert and delete anomalies each step removes. Two choices were deliberate: a generated column for a computed total, and stored `seats taken` counter on course offerings, kept denormalised so seat checks stay fast.

Portfolio 2: Query processing and optimization. I seeded 50,000 students, 250,000 enrolments and 208,458 results, then measured three queries with `EXPLAIN ANALYZE` before and after optimisation.

Query	Before	After
A. Students by surname	Seq Scan, 3.3 to 11.6 ms	Bitmap Heap Scan using index, about 0.5 ms
B. Enrolments for one offering	Parallel Seq Scan, about 235 ms	Bitmap Heap Scan using index, about 1.2 ms
C. Top courses by average score	about 630 ms	about 0.66 ms using a materialised view

Indexes cost storage (368 kB and 1,776 kB here) and add a little work to every write. The materialised view is very fast but is only as fresh as its last refresh.

Portfolio 3: Transactions and concurrency. I demonstrated commit and rollback, and a seat-booking race. Under PostgreSQL's default READ COMMITTED isolation level, two sessions could both read that one seat remained and both enrol, overselling the course. Locking the course offering row with `SELECT ... FOR UPDATE` made the second session wait, re-read the updated seat count, and be correctly refused.

Portfolio 4: NoSQL and advanced data models. LMS activity events are high-volume and differ in shape by event type, so I stored 100,000 of them as documents in MongoDB Atlas (`mivadb_logs.lms_activity`), linked to PostgreSQL by `student_id`. A student lookup without an index scanned all 100,000 documents in 89 ms. A compound index on `(student_id, timestamp)` reduced this to 4 documents examined in 1 ms. Aggregation queries over the whole collection took 254 to 432 ms, because an index on one student does not help queries that read most of the data.

Portfolio 5: Distributed and cloud database. Using Docker Compose, I ran a PostgreSQL 16 primary and a streaming replica. `pg_stat_replication` showed the replica as `streaming` in `async` mode. A table created and filled with the primary appeared on the replica, and an insert sent directly to the replica was refused as read-only. Because replication is asynchronous, the replica can lag and the latest writes could be lost if the primary crashed suddenly. Replicas scale reads, not writes. For sharding, splitting students by cohort keeps cohort queries on one server, but the newest cohort becomes a hot spot and reports across cohorts must query every shard. At MivaDB's scale, I conclude that indexing, one primary with a replica and partitioning inside a single server are enough, and sharding should wait until measurements show one primary cannot cope.

3. Where the evidence is in the repository

Folder	Key evidence
`portfolio-1-database-design`	`schema.sql`, `er-diagram.png`, `normalisation.md`
`portfolio-2-query-optimisation`	`seed.py`, README comparison table, execution plan screenshots
`portfolio-3-transactions-concurrency`	`transactions.sql`, README, screenshots of both sessions
`portfolio-4-nosql`	`seed_mongo.py`, `queries_mongo.py`, Compass and terminal screenshots
`portfolio-5-distributed-cloud`	`docker-compose.yml`, `pg_hba.conf`, `init-primary.sql`, replication screenshots

Each folder has a README explaining how the work was implemented and tested, and the main README links all five. The commit history shows the work built up progressively from 23 October to 5 November 2026.

4. Individual technical reflection

Design choices. I used PostgreSQL for the structured academic records because they need constraints, foreign keys and transactions, and used MongoDB only for LMS events, whose fields differ by type. I accepted one denormalised column and one materialised view because the measurements showed recalculating was costly. I used Docker for Portfolio 5 so I could set up and observe replication directly instead of relying on a managed service that hides it.

Challenges. MongoDB Atlas would not connect from my laptop and showed an SSL error. The real cause was that my IP address was not on Atlas's network access list, so I learned that an error message can point to the wrong layer. I also moved the Atlas connection string out of my code into an environment variable so GitHub doesn't store a password. A push once failed due to a network problem, and I learned to confirm every push with `git status` and by checking the repository online.

Lessons learned. Measuring before and after is what makes optimisation credible: the same index that cut one lookup from 89 ms to 1 ms did nothing for whole-collection analysis. No single database suits every kind of data. Replication improves availability and read capacity but not write capacity, and it brings a consistency trade-off. Complexity such as sharding should be added only when measurements justify it.

Limitations. The data is randomly generated, so values such as average quiz scores demonstrate that the queries work, not real student behaviour. Timings are single runs and will vary. I did not measure replication lag or automate failover.

5. AI use declaration

- **AI tool used:** Claude
- **What it was used for:** planning the portfolio, explaining concepts, helping to write and debug scripts, and drafting parts of the README and documentation text.
- **How I reviewed and verified the output:** I ran every script myself against my own PostgreSQL, MongoDB Atlas and Docker setups. All results, timings and screenshots come from my own runs. I fixed the errors that came up, checked outputs against my understanding of the concepts, and edited the text. I am responsible for and can explain, everything submitted.

6. References

- PostgreSQL Global Development Group (n.d.) *EXPLAIN*. PostgreSQL Documentation. Available at: https://www.postgresql.org/docs/current/sql-explain.html
- PostgreSQL Global Development Group (n.d.) *Table Partitioning*. PostgreSQL Documentation. Available at: https://www.postgresql.org/docs/current/ddl-partitioning.html
- MongoDB, Inc. (n.d.) *Indexes*. MongoDB Manual. Available at: https://www.mongodb.com/docs/manual/indexes/
- MongoDB, Inc. (n.d.) *PyMongo Documentation*. Available at: https://pymongo.readthedocs.io
- Docker, Inc. (n.d.) *Docker Compose documentation*. Available at: https://docs.docker.com/compose/
- Faker (n.d.) *Faker documentation*. Available at: https://faker.readthedocs.io